

Chapter 07: Advanced Data Structures: Segment Trees, Fenwick Trees, & Bitmask DP

Zero-Prerequisite Intuition: The "Tournament Bracket" Metaphor Why do we need Segment Trees and Fenwick Trees? Imagine an array of 1,000,000 numbers. Your manager gives you two operations:

1. "Update index 42,000 to value 99."
2. "Calculate the sum of all numbers between index 10,000 and 800,000."

- **Naive Array:** Operation 1 is instant ($O(1)$), but Operation 2 requires summing 790,000 numbers in a loop ($O(N)$). If you do this 100,000 times, your code times out!
- **Prefix Sum Array:** Operation 2 is instant ($O(1)$ subtraction: `prefix[800000] - prefix[10000]`), but Operation 1 requires recalculating 960,000 prefix sums ($O(N)$)!

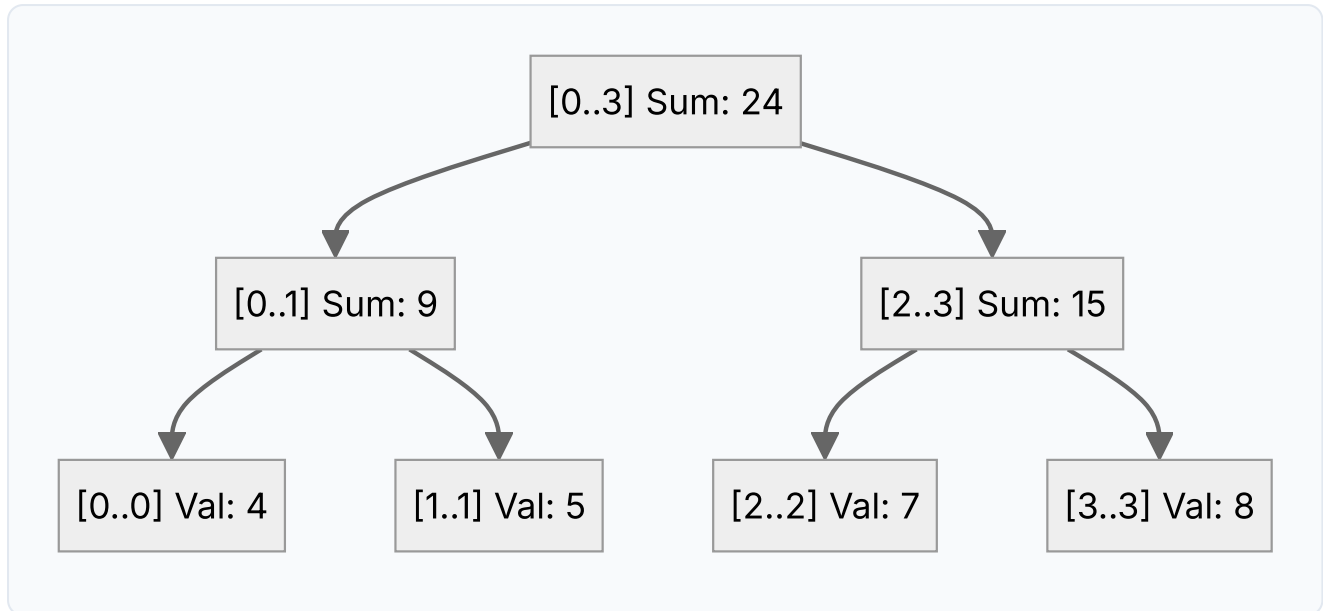
You are trapped: either queries are slow, or updates are slow.

How do we make BOTH Updates and Queries blazing fast in $O(\log N)$ time? Imagine a **Tennis Tournament Bracket**. Two players play in Round 1. The winner moves to Round 2. The winner moves to the Semifinals, and the champion is at the top. If one player withdraws and is replaced, you don't replay the entire tournament! You only re-evaluate the **single branch** of matches leading from that player up to the final ($\log_2 N$ matches).

That is a **Segment Tree**.

1. Segment Trees: Range Queries & Point Updates in $O(\log N)$

A Segment Tree is a binary tree where every node represents an interval `[L, R]`. Leaf nodes represent single array elements. Parent nodes store the aggregate (Sum, Min, or Max) of their children.



Complete Segment Tree Implementation in Python

2. Fenwick Tree (Binary Indexed Tree / BIT)

A Fenwick Tree achieves the exact same $O(\log N)$ updates and prefix sum queries as a Segment Tree, but uses **half the memory** and **only 12 lines of code!**

The Magic of the Lowest Set Bit: $i \& (-i)$

In two's-complement binary arithmetic, $i \& (-i)$ isolates the lowest 1-bit of integer i . Each index i in a Fenwick tree stores the sum of a range of length $i \& (-i)$.

3. Disjoint Set Union (Union-Find) with Rank & Path Compression

DSU tracks a partition of elements into disjoint subsets.

- **Path Compression:** Flattens the tree during `find()`, making all nodes point directly to the root.
- **Union by Rank:** Attaches the shallower tree under the deeper tree to prevent tall degenerated linked lists.

- **Time Complexity:** The Inverse Ackermann function $\alpha(N) \leq 4$ for any $N \leq 10^{80}$ (the number of atoms in the observable universe!). It is **effectively** $O(1)$ in practice.

4. Bitmask Dynamic Programming

When an interview problem involves visiting all subsets of items (like the Traveling Salesperson Problem or assigning N jobs to N workers), a naive permutation takes $O(N!)$ time ($15! \approx 1.3 \times 10^{12}$ operations, which will crash).

Bitmask DP represents subsets as integers where the i -th bit indicates whether item i is included (1) or excluded (0).

The Traveling Salesperson Problem (TSP) in $O(N^2 2^N)$

This reduces $O(N!)$ time down to $O(N^2 2^N)$, making problems up to $N = 20$ solvable in milliseconds!

Further Reading

- [CP-Algorithms: segment tree](#)
- [CP-Algorithms: Fenwick tree](#)
- [CP-Algorithms: DSU](#)

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. What does a segment tree support?

Range queries and point or range updates in $O(\log n)$.

▼ 2. Fenwick tree versus segment tree?

Fenwick (BIT) is simpler and smaller for prefix sums with point updates; a segment tree is more general (range min/max, lazy updates).

▼ 3. When use bitmask DP?

When the state is a subset of a small set (n up to about 20), e.g. TSP-style problems.

▼ 4. What is the complexity of union-find with path compression and union by rank?

Nearly constant per operation (inverse Ackermann).